

SimpleNOC Architecture

System Design, Data Flow Pipelines & Technical Interdependency Specification

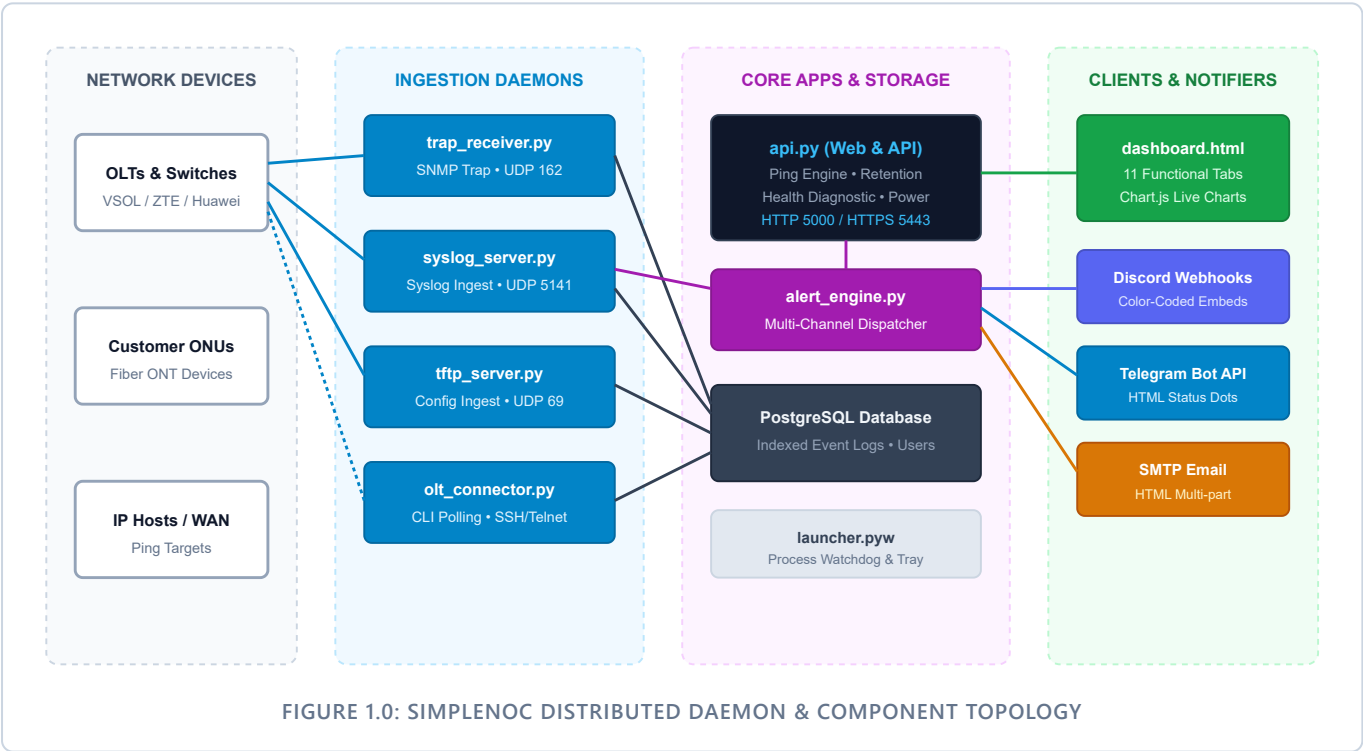
RELEASE V0.5.6.4

TARGET PLATFORM	DATABASE	PRIMARY RUNTIME	ARCHITECTURE TYPE
Windows 10 / 11 / Server	PostgreSQL 12+	Python 3.10+ / Flask	Decoupled Micro-Daemons

1. Executive Summary & Product Purpose

SimpleNOC is a high-availability, modular Network Operations Center (NOC) platform tailored for Internet Service Providers (ISPs), fiber broadband operators, and Optical Line Terminal (OLT) networks. The platform operates 24x7 in headless or desktop mode, continuously aggregating asynchronous network telemetry, tracking device health, providing in-app power lifecycle recovery, and dispatching real-time notifications to network engineers.

Key Architectural Goal: Provide single-pane-of-glass operational visibility across multi-vendor OLTs (VSOL, ZTE, Huawei, Fiberhome), syslog streams, SNMP traps, and ping targets with zero downtime, instant color-coded alerting, and complete in-app self-healing recovery.



2. Component Catalog & Interdependency Matrix

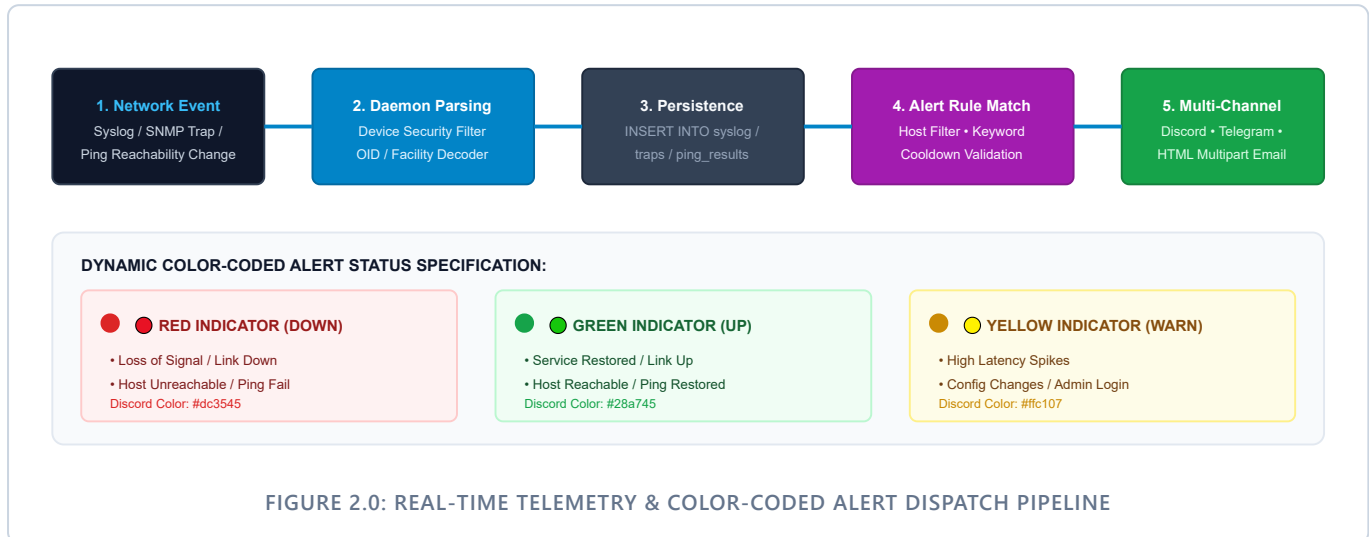
All daemons run concurrently as independent processes coordinated by the central supervisor (`launcher.pyw`) and communicate through the PostgreSQL database layer and loopback REST calls.

Component File	Protocol / Port	Primary Functional Responsibilities	Interlinked Dependencies
noc_config.py	Internal	Central configuration repository; DB connection pooling (query_db, execute_db); encryption keys; default ports & retention parameters.	Imported by all modules (api.py, alert_engine.py, trap_receiver.py, syslog_server.py, tftp_server.py, launcher.pyw).
launcher.pyw	Desktop GUI	Desktop process supervisor; tray controller; service heartbeat monitor; watchdog auto-restart for hanging processes.	Spawns & monitors api.py, trap_receiver.py, syslog_server.py, tftp_server.py; polls HTTP /api/health.
api.py	HTTP 5000 HTTPS 5443	Flask REST API core; session auth; Ping Engine; Retention Cleanup; Health Diagnostics; Power Management (Restart/Shutdown).	Serves dashboard.html, login.html; triggers alert_engine.py on ping changes; calls olt_connector.py.
dashboard.html	Client UI	Single-page app with 11 functional tabs; Chart.js visual graphs; drag-drop tab order; power control modals; diagnostic verdict banner.	Communicates via authenticated JSON REST calls with api.py; displays real-time telemetry from all daemons.
alert_engine.py	Outbound REST/SMTP	Rule matcher; visual status renderer (● Red / ● Green / ● Yellow); Discord embed creator; Telegram bot dispatcher; HTML email sender.	Invoked by syslog_server.py, trap_receiver.py, and api.py; logs dispatched alerts to PostgreSQL alert_log table.
syslog_server.py	UDP 5141	RFC 3164/5424 syslog listener; Device Registration gatekeeper (Allow/Deny); persists to syslog table; triggers alerts.	Uses noc_config.py for DB; calls alert_engine.process_syslog_alert().
trap_receiver.py	UDP 162	SNMP v1/v2c trap receiver; PySNMP decoder; OID translation via vsol_mib.py; persists to traps table.	Uses vsol_mib.py for OID names; uses noc_config.py for PostgreSQL persistence.
tftp_server.py	UDP 69	TFTP upload server for OLT and switch configuration files; path-traversal sanitization; indexes metadata in tftp_files.	Writes config backups to /backups/; updates PostgreSQL tftp_files.
olt_connector.py	SSH / Telnet	Paramiko automation library for VSOL, ZTE, Huawei, and Fiberhome OLTs; extracts ONUs, optical power (Rx/Tx), and uplink traffic.	Invoked on-demand by api.py and background scheduler; updates onu_data and olt_data.

3. Core Data Flow & Processing Pipelines

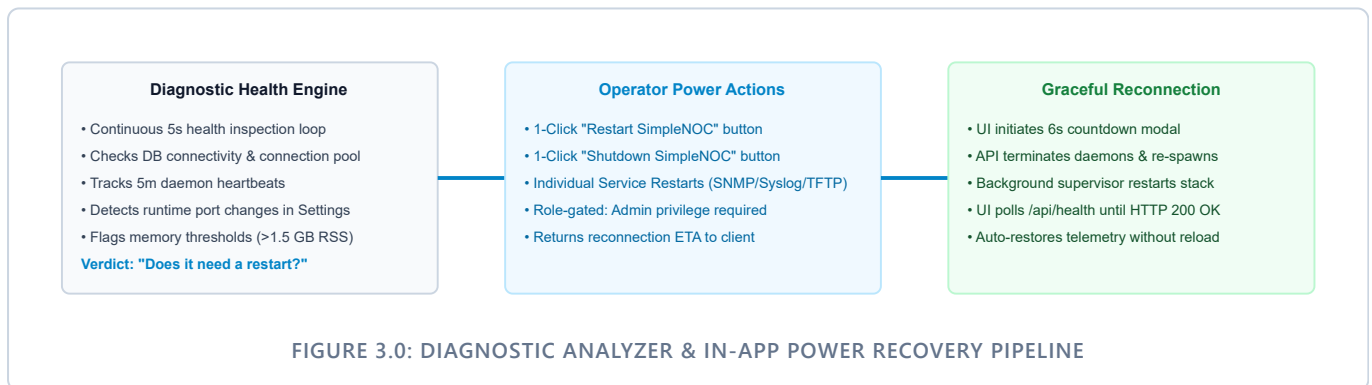
3.1. Telemetry Ingestion & Color-Coded Alert Pipeline

When network events arrive, the platform analyzes device authorization, extracts structured variables, records data into indexed PostgreSQL tables, and evaluates alert rules with instant color-coded visual status indications.



3.2. Diagnostic Health & In-App Power Recovery Pipeline

SimpleNOC incorporates an automated diagnostic evaluation engine alongside full in-app power lifecycle management (`/api/system/restart`, `/api/system/shutdown`, and `/api/system/service_action`) so operators can reboot the stack without host terminal access.



4. PostgreSQL Database Schema Specification

The PostgreSQL database is organized into normalized relational tables optimized for time-series ingestion and fast indexed lookups.

Table Name	Primary Columns	Operational Purpose & Indexing Strategy
users	id, username, password, salt, role, visible_tabs, assigned_olts	Stores credentials hashed via PBKDF2-HMAC-SHA256 (200k rounds) with salt; handles role-based permissions (admin vs viewer) and tab access lists.
syslog	id, timestamp, host, facility, severity, tag, message	High-throughput incoming syslog event store. Indexed on timestamp and host for real-time dashboard searches. Automatic truncation at 150 MB.
syslog_devices	id, device_ip, hostname, olt_mac, authorized, created_at	Security gatekeeper table. Controls device ingestion access (authorized=1 for allow, 0 for deny/unregistered).
traps	id, timestamp, source_ip, trap_oid, enterprise_oid, details, severity	Decoded SNMP v1/v2c traps with human-readable variable bindings translated from VSOL/standard enterprise MIBs.
alert_rules	id, name, host, exclude_hosts, contains_text, notify_via, webhook_url	Configurable alert rule conditions matching syslog keywords, ping down events, severity levels, and notification routes.
alert_log	id, timestamp, rule_name, host, message, channel, status	Historical audit log of all dispatched alerts across Discord, Telegram, and Email channels.
ping_targets	id, name, ip, group_name, interval_sec, status, last_rtt	Ping monitor inventory tracking host IP, reachability state (ONLINE / OFFLINE), and round-trip latency (ms).
olt_profiles	id, name, ip, vendor, protocol, username, password, snmp_community	Credentials and connection parameters for automated OLT SSH/Telnet polling.
onu_data	id, olt_ip, slot_port, onu_id, mac, online, rx_power, tx_power, last_seen	Live and historical inventory of discovered customer ONUs, optical attenuation levels, and authorization states.
tftp_files	id, filename, filepath, client_ip, filesize, upload_time, checksum	Inventory of configuration backups uploaded by network switches and OLTs via the TFTP daemon.

5. Security & Directory Architecture

```
e:\ntigravity|— api.py                # Flask Web & REST API Core Backend (HTTP 5000 / HTTPS 5443)
|— launcher.pyw                      # Supervisor GUI & Watchdog Process (Tkinter)
|— alert_engine.py                   # Multi-Channel Alert & Rule Dispatcher (Discord/Telegram/Email)
|— syslog_server.py                  # UDP 5141 Syslog Listener Daemon (Device Registration Gatekeeper)
|— trap_receiver.py                  # UDP 162 SNMP Trap Listener Daemon (PySNMP & MIB Decoder)
|— tftp_server.py                    # UDP 69 TFTP Backup Storage Daemon (Path Traversal Sanitizer)
|— olt_connector.py                  # SSH/Telnet OLT & ONU Polling Engine (ZTE/VSOL/Huawei/Fiberhome)
|— vsol_mib.py                       # Enterprise MIB OID Dictionary & Translator
|— noc_config.py                     # Central Application Settings & PostgreSQL Connection Pool
|— gen_cert.py                       # 2048-bit RSA Self-Signed TLS/SSL Certificate Generator
|— check_downtime.py                 # Historical Service Uptime Gap Audit Tool
|— setup.py                          # Windows Installation & Service Configuration Engine
|— init_postgres.sql                 # PostgreSQL Base Database Initialization Script
|— dashboard.html                    # Single Page Web App (11 Modular Tabs & Chart.js Visualizations)
|— login.html                        # Secure Authentication Frontend with PBKDF2 Password Hashing
|— backups/                          # TFTP Uploads & Switch Configuration Snapshots
|— logs/                             # Daemon Heartbeats & Runtime Logs (API, SNMP, Syslog, TFTP)
```